

Lecture 9: Outline

- ▶ Minimization and analysis.
 - ▶ characterization of minima (extreme values)
 - ▶ connection with linear systems
 - ▶ connection with definite matrices.

Lecture 9: The chief problem

- ▶ Minimization of multivariable functions
 - ▶ Challenges:
 - ▶ really a lot of variables,
 - ▶ complicated (long) functions, possibly a number of local extreme values.
 - ▶ In neural networks: variables – parameters (frequently without meaning).
 - ▶ Task in general: find parameters in a neural network such that its loss is minimal.
- ▶ Is there a minimum (minimal loss)?
- ▶ How can we find the variables/parameters leading to this minimum?
- ▶ We have real chance only with numerical methods.

Lecture 9: The function to minimize in details

- ▶ In a general form: we have $\Phi : \mathbb{R}^N \rightarrow \mathbb{R}$ to minimize.
- ▶ In more details: $\Phi(\mathbf{s}) = L(NN(\mathbf{s}, \mathbf{w}_{\text{in}}), \mathbf{w}_{\text{out}})$, where
 - ▶ $\mathbf{w}_{\text{out}}$: given output – usually a large number of vectors.
 - ▶ $\mathbf{w}_{\text{in}}$: given input – usually a large number of vectors.
 - ▶ $\mathbf{s} \in \mathbb{R}^N$: parameter-vector of the neural network
 - ▶ variables to minimize,
 - ▶ responsible for complexity.
 - ▶ NN : the neural network itself
 - ▶ Including final manipulations (e.g., rounding, one-hot decoding)
 - ▶ depends on the structure of the network.
 - ▶ L : loss function
 - ▶ a kind of distance between the computed output $NN(\mathbf{s}, \mathbf{w}_{\text{in}})$ and the real one $\mathbf{w}_{\text{out}}$.

Lecture 9: An example to imagine

- ▶ $\mathbf{w}_{\text{in}} = [W_1, W_2, \dots, W_{10^4}]$ and $W_i \in \mathbb{R}^{480 \times 640}$,
- ▶ $\mathbf{w}_{\text{out}} = [\mathcal{W}_1, \mathcal{W}_2, \dots, \mathcal{W}_{10^4}]$ such that each $\mathcal{W}_i \in \mathbb{R}^{10}$.
- ▶ $\mathbf{s} = [A_1, \mathbf{b}_1, A_2, \mathbf{b}_2]$, where
 - ▶ $A_1 \in \mathbb{R}^{100 \times 480 \cdot 640}$, $\mathbf{b}_1 \in \mathbb{R}^{100}$, $A_2 \in \mathbb{R}^{10 \times 100}$, $\mathbf{b}_2 \in \mathbb{R}^{10}$
 - ▶ a number of $\approx 10^6$ variables
 - ▶ of course, in real life, mostly simple matrix structures are used
- ▶
$$\Phi(\mathbf{s}) = \sum_{j=1}^{10^4} \|(A_2(A_1 W_j + \mathbf{b}_1) + \mathbf{b}_2) - \mathcal{W}_j\|^2$$

Lecture 9: Further details on real-life situations

- ▶ Regarding the input data:
 - ▶ The $\mathbf{w}_{in}$ input can be a 3-dimensional array consisting of matrices (pictures).
 - ▶ Filtering/cleaning of data and a kind of normalization is usually necessary.
- ▶ Regarding the number of variables:
 - ▶ One should reduce it; not only for reducing computational time,
 - ▶ but rather to avoid “overfitting”.
 - ▶ Try to use a “clever” network, which has some meaning, background;
 - ▶ Try to use convolutional layers instead of dense ones.

Lecture 9: Minimization - theory

- ▶ If Φ is differentiable and has a minimum at $\mathbf{s}_0$
 - ▶ then $\partial_1\Phi(\mathbf{s}_0) = \partial_2\Phi(\mathbf{s}_0) = \dots = \partial_N\Phi(\mathbf{s}_0) = 0$, i.e. $\nabla\Phi(\mathbf{s}_0) = \mathbf{0}$.
 - ▶ In principle: N equations and N unknowns.
 - ▶ In practice: some difficulties with this system
 - ▶ Usually, non-linear and existence of a unique solution is not guaranteed.
 - ▶ Even if we could solve it: The function Φ may also have a local maximum at $\mathbf{s}$ if

$$\partial_1\Phi(\mathbf{s}_0) = \partial_2\Phi(\mathbf{s}_0) = \dots = \partial_N\Phi(\mathbf{s}_0) = 0 \Leftrightarrow \nabla\Phi(\mathbf{s}_0) = \mathbf{0}. \quad (1)$$

Lecture 9: Minimization - theory

- ▶ If (1) is valid, $\Phi(\mathbf{s}_0)$ is twice differentiable and $\Phi''(\mathbf{s}_0) \in \mathbb{R}^{N \times N}$ has only positive eigenvalues, then Φ has a local minimum at $\mathbf{s}_0$.
 - ▶ This is the straightforward generalization of the 1D theory.
 - ▶ Even now, we can only ensure local minimum.
- ▶ Another idea: Solve (1) by using a Newton method.
 - ▶ This needs the derivative of $\nabla\Phi$ at some points, which is again $\Phi''(\mathbf{s}_0)$.
 - ▶ This is a matrix of size $N \times N$.
 - ▶ We have to compute all of its elements by computing a possibly long differentiation of composition functions. Say, we are very clever and can solve it using only 5 function evaluations.
 - ▶ Even in this case, we need $5N^2$ evaluations, which would be too much.

Lecture 9: Minimization - theory

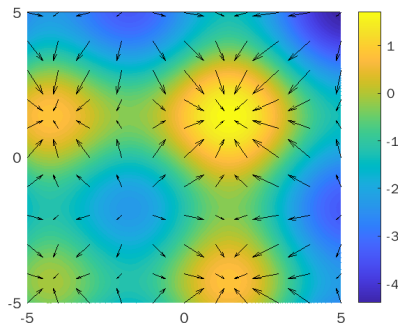
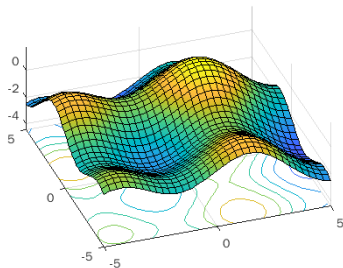
- ▶ Is it sure that we have a minimum?
 - ▶ Important: anyway a numerical method may not converge anywhere.
- ▶ If Φ is continuous everywhere and we have a condition $\mathbf{s} \in [-M, M]^n$ for the parameters, then for sure, Φ should have a minimum somewhere.
 - ▶ Conditions here are reasonable.
- ▶ Summarized: we have a minimum but using analysis, it is usually hard to find it.
- ▶ For an efficient algorithm some interpretation and further analysis.

Lecture 9: Minimization - preparing a numerical method

- ▶ Clear geometrical interpretation for $N = 2$.
 - ▶ Graph of Φ is a surface over the parameter-plane.
 - ▶ We need to find parameters for the downmost point of the surface.
 - ▶ For this, we need to move downwards along/on the surface.
 - ▶ To be efficient, move downwards as steep as possible.
 - ▶ At a certain point $\mathbf{s}$, which direction corresponds to the steepest descent (motion downwards)?
 - ▶ Answer from the theory: this corresponds to $-(\partial_1 \Phi(\mathbf{s}), \partial_2 \Phi(\mathbf{s}), \dots, \partial_N \Phi(\mathbf{s})) = -\nabla \Phi(\mathbf{s})$.
 - ▶ This defines us a direction; the length should be chosen carefully.

Lecture 9: Find a local maximum - geometrical interpretation

- ▶ A surface corresponding to a function $\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}$ (left), its top view and some of its gradients (right).
- ▶ Here, maximization is investigated.



Lecture 9: Minimization - problems in the above sketch

- ▶ This can lead only to a local minimum.
 - ▶ If the surface has much local minima, we need extra work to try find the real one.
 - ▶ To approach the “real one”, one needs to start parameters from the close neighborhood of the real minimum.
- ▶ This needs the computation of derivatives quickly.
 - ▶ Python has an efficient built-in procedure to do it, if NN is composed of standard components.
 - ▶ There are also “gradient-free” mutants of this procedure.